

CHANDIGARH UNIVERSITY

Discover. Learn. Empower.

PRACTICAL FILE

on

PERSONAL EXPENSE TRACKER

using Oracle APEX and PL/SQL

PL/SQL | Subject Code: 26CAP-602

Submitted By

Name: UJJAL ROY

UID: 26MCA20159

Class: 26MCA-3

Section: A

Submitted To

Name: TANISHA RANI

Designation: Professor

Ecode: E20679

Course: PL/SQL

Master of Computer Applications (MCA)
University Institute of Computing
Chandigarh University
Gharuan, Mohali, Punjab, India - 140413
October 2026

Abstract

Personal Expense Tracker is a small database application for recording expenses and reviewing them by month. The database is implemented in Oracle SQL and PL/SQL. The user-facing interface is built in Oracle APEX and provides an Add Expense form and an Expense History report. The data model separates expense categories from expense records and links them with a foreign key.

The database script creates the two tables, inserts seven sample categories and thirteen sample expenses, and defines the ADD_EXPENSE procedure and GET_MONTH_TOTAL function. The procedure validates amount, date, category and description length before inserting an expense. The function returns the sum for a selected calendar month. The APEX history page joins expenses to categories, displays the selected month total and lists matching records.

The supplied application export identifies Oracle APEX 26.2. The public application was also opened through the submitted URL; the visible page showed the visitor as "nobody." The supplied October 2026 screenshot shows eight rows and a total of 7,133.75 after a further 150.00 lunch expense had been added to the original sample data.

The project demonstrates relational tables, keys, check constraints, stored procedures, a stored function, exception messages, date filtering, aggregate functions and a browser-based interface. It is a learning demonstration rather than a private financial service: the application has no user-specific authentication on its public pages, and entries share the same underlying data.

Acknowledgement

I thank Ms. Tanisha Rani, Professor, for the course instruction and project direction. I also acknowledge the Oracle APEX learning environment and the Oracle documentation used to understand the database and application features.

Keywords: Oracle APEX, PL/SQL, expense tracking, stored procedure, stored function, relational database, validation.

Contents

Report structure and figure index

Section	Page
Abstract and Acknowledgement	2
Introduction and Problem Definition	4
Objectives, Scope and Limitations	5
Tools, Architecture and Workflow	6
Database Design	7
PL/SQL Implementation	8
Oracle APEX Implementation	9
Application Screenshots	10
Testing and Results	11
User Guide and Constraints	12
Conclusion and References	13

List of Figures

Figure 1. Relationship between CATEGORIES and EXPENSES (page 7)

Figure 2. Add Expense form with a negative value entered before submission (page 10)

Figure 3. Expense History for October 2026 (page 10)

Abbreviations

Term	Meaning
APEX	Oracle Application Express
DBMS	Database Management System
FK / PK	Foreign Key / Primary Key
PL/SQL	Procedural Language extension to SQL
SQL	Structured Query Language

Chapter 1: Introduction

1.1 Background

Small daily purchases can be difficult to review when they are remembered informally or written in scattered notes. A database can keep each expense as a structured record with an amount, category, date and short description. Once records are stored consistently, a query can list them and calculate totals for a time period.

This project uses Oracle Database for storage and PL/SQL for server-side operations. Oracle APEX supplies browser pages that call database logic and show results. The design keeps the key rules close to the data: the amount must be positive, every expense must reference a category, and required date information must be present.

1.2 Problem definition

The project addresses a simple task: record an expense and later review the records for a chosen month. A form alone would collect input but would not enforce all database rules. A report alone would display data but would not provide a controlled way to add it. The application therefore combines a form, a PL/SQL validation procedure, relational constraints and a monthly history report.

1.3 Proposed solution

The user enters an amount, chooses one of seven categories, selects a date and optionally enters a short description. APEX passes those values to ADD_EXPENSE. If they pass validation, the procedure inserts the record and commits it. The history page uses the month selection to show matching expenses and a total computed by GET_MONTH_TOTAL.

1.4 Report scope

This report explains the supplied SQL script, APEX application export, observed public pages and user-provided test notes. It describes the schema and program logic and includes actual application screenshots. It does not claim deployment to a commercial service, isolated accounts, or features that are not present in the supplied implementation.

Chapter 1: Objectives and Scope

1.5 Objectives

- Create category and expense tables with primary-key and foreign-key relationships.
- Prevent invalid expense amounts at both table and procedure level.
- Validate the expense date, category selection and description length before saving.
- Provide a reusable monthly total function using Oracle date operations.
- Build APEX pages for entering an expense and reviewing monthly history.
- Show the saved records and monthly total in a browser-based interface.

1.6 Functional scope

Included	Not included in this version
Create an expense with amount, category, date and description.	Edit or delete an existing expense.
Seven predefined expense categories.	User-defined categories or category maintenance.
List expenses for a selected month.	Charts, budgets, export and notifications.
Calculate the total for the selected month.	Separate accounts or private per-user records.

1.7 Intended use

The application is suitable as a classroom demonstration of database-backed form processing. The public APEX URL allows a visitor to open the application, but the shared, unauthenticated pages are not appropriate for personal or sensitive financial records. Test entries may be visible to other visitors and affect the common monthly total.

1.8 Success criteria

The project is considered successful when the SQL objects can be created, an accepted expense appears in the report, invalid data is rejected, and the selected-month total matches the visible records. The supplied screenshots demonstrate the public entry form and an October history view; the reported SQL execution and negative-amount behavior are recorded in Chapter 4.

Chapter 2: Tools and System Design

2.1 Tools and environment

Component	Role in the project
Oracle Database	Stores categories and expense rows and executes SQL/PL/SQL.
Oracle APEX 26.2	Hosts the form and monthly history pages in a browser.
SQL Workshop	Runs the schema, seed data, procedure and function script.
App Builder / Page Designer	Defines APEX pages, items, report region and page processes.
Google Chrome on macOS	Browser used for the supplied application screenshots.

2.2 Three-layer view

Layer	Project element	Responsibility
Presentation	Oracle APEX pages	Collect input and display history.
Logic	ADD_EXPENSE / GET_MONTH_TOTAL	Validate inserts and calculate monthly totals.
Data	CATEGORIES / EXPENSES	Persist categories and expense records.

2.3 Request flow

A user submits the Add Expense page. APEX page items supply values to the ADD_EXPENSE process. The procedure checks the inputs, writes one row and commits it when valid. On the history page, the selected month is supplied to GET_MONTH_TOTAL, while a report query filters expense dates and joins the category name.

2.4 Data integrity

The schema uses identity-generated primary keys, a unique category name, a positive-amount check constraint and a category foreign key. Procedure checks provide clearer messages for several invalid inputs. Database constraints remain useful as a final safeguard if data is inserted through a different route.

Chapter 3: Database Design

3.1 Entity relationship

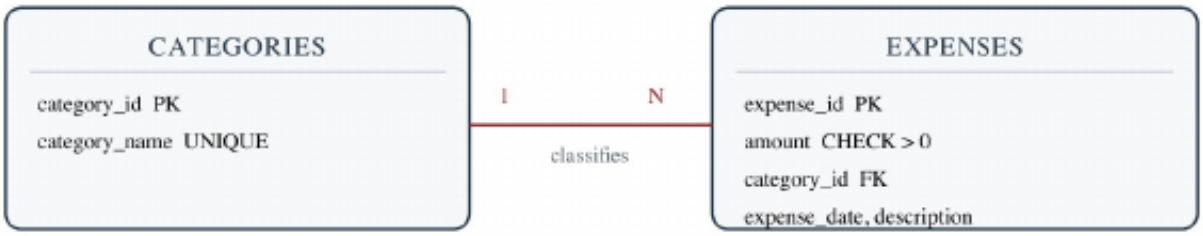


Figure 1. Logical relationship implemented in the SQL script.

Table / column	Type / rule	Purpose
CATEGORIES.category_id	NUMBER, identity, primary key	Unique identifier for a category.
CATEGORIES.category_name	VARCHAR2(50), NOT NULL, UNIQUE	Category label such as Food or Rent.
EXPENSES.expense_id	NUMBER, identity, primary key	Unique identifier for an expense.
EXPENSES.amount	NUMBER(10,2), NOT NULL, CHECK > 0	Positive expense amount with two decimal places.
EXPENSES.category_id	NUMBER, NOT NULL, foreign key	References CATEGORIES.category_id.
EXPENSES.expense_date	DATE, NOT NULL	Date on which the expense occurred.
EXPENSES.description	VARCHAR2(200)	Optional short description.

3.2 Relationship and normalization

Each category is stored once, while many expense rows can refer to that category. This avoids repeating category text in every record and provides a referential integrity rule. The APEX report joins the tables when it needs to display a readable category name.

Chapter 3: PL/SQL Implementation

3.3 ADD_EXPENSE procedure

ADD_EXPENSE centralizes checks before writing a row. It rejects a null or non-positive amount, a missing date, a future date, a category ID that does not exist, and a description longer than 200 characters. For a rejected input it raises an application error with a code from -20001 through -20005. A valid date is truncated to midnight before insertion.

Validation excerpt from the supplied PL/SQL script:

```
IF p_amount IS NULL OR p_amount <= 0 THEN
  RAISE_APPLICATION_ERROR(-20001,
    'Amount must be greater than zero.');
END IF;

IF p_expense_date IS NULL THEN
  RAISE_APPLICATION_ERROR(-20002,
    'Please enter the expense date.');
END IF;

IF p_expense_date > TRUNC(SYSDATE) THEN
  RAISE_APPLICATION_ERROR(-20003,
    'Date cannot be in the future.');
END IF;

-- Check that the selected category exists.
SELECT COUNT(*) INTO v_count
FROM categories
WHERE category_id = p_category_id;

IF v_count = 0 THEN
  RAISE_APPLICATION_ERROR(-20004,
    'Please choose a valid category.');
END IF;
```

3.4 Insert and commit

After all checks pass, the procedure inserts amount, category, date and description into EXPENSES and commits the transaction. This means a valid submission is immediately persisted. The commit behavior also means the caller cannot later roll back this specific insert in the same transaction.

3.5 Error handling

RAISE_APPLICATION_ERROR sends a controlled application error to the caller instead of silently accepting an invalid row. The APEX process can display the message to the user. The table-level CHECK constraint independently prevents a zero or negative amount even if an insert bypasses the procedure.

Chapter 3: Oracle APEX Implementation

3.6 Add Expense page

The exported APEX application is named Personal Expense Tracker. Its Add Expense page contains fields for amount, category, date and short description. The page process calls ADD_EXPENSE when the user submits the form. The APEX application export marks the user-facing pages as public, and the supplied public URL opened without a sign-in prompt.

3.7 Expense History page

The Expense History page provides a month selector and a total item, followed by a report of expenses. The report query joins EXPENSES to CATEGORIES so that category names are displayed. The visible capture is sorted with the most recent date first and lists eight records for October 2026.

3.8 GET_MONTH_TOTAL function

The function accepts any date within a target month. TRUNC(p_month_date, 'MM') finds the first day of that month. ADD_MONTHS computes the first day of the next month. The half-open range includes all times on the first day and excludes the next month. NVL returns zero if no rows match.

Monthly total excerpt from the supplied function:

```
SELECT NVL(SUM(amount), 0)
  INTO v_total
  FROM expenses
 WHERE expense_date >= TRUNC(p_month_date, 'MM')
   AND expense_date <  ADD_MONTHS(
         TRUNC(p_month_date, 'MM'), 1);

RETURN v_total;
```

3.9 Public access consideration

The public pages make it easy to demonstrate the application. They also use shared database data. A visitor who can reach the Add Expense form may affect the shared history. The application should therefore be treated as a public classroom demonstration and should not be used for real private records.

Chapter 4: Application Screenshots

Actual screenshots supplied by the student, captured in Oracle APEX.

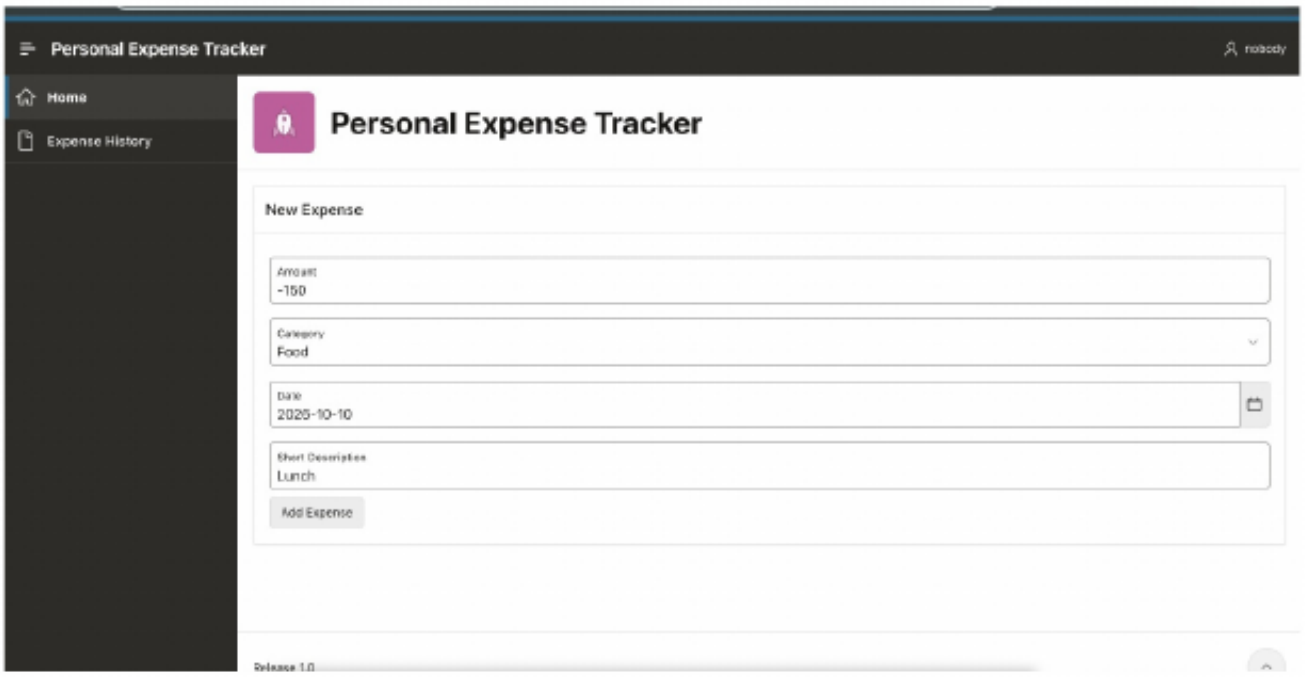


Figure 2. Add Expense form with a negative amount entered before submission. The capture shows the input state; it does not show the post-submit validation message.

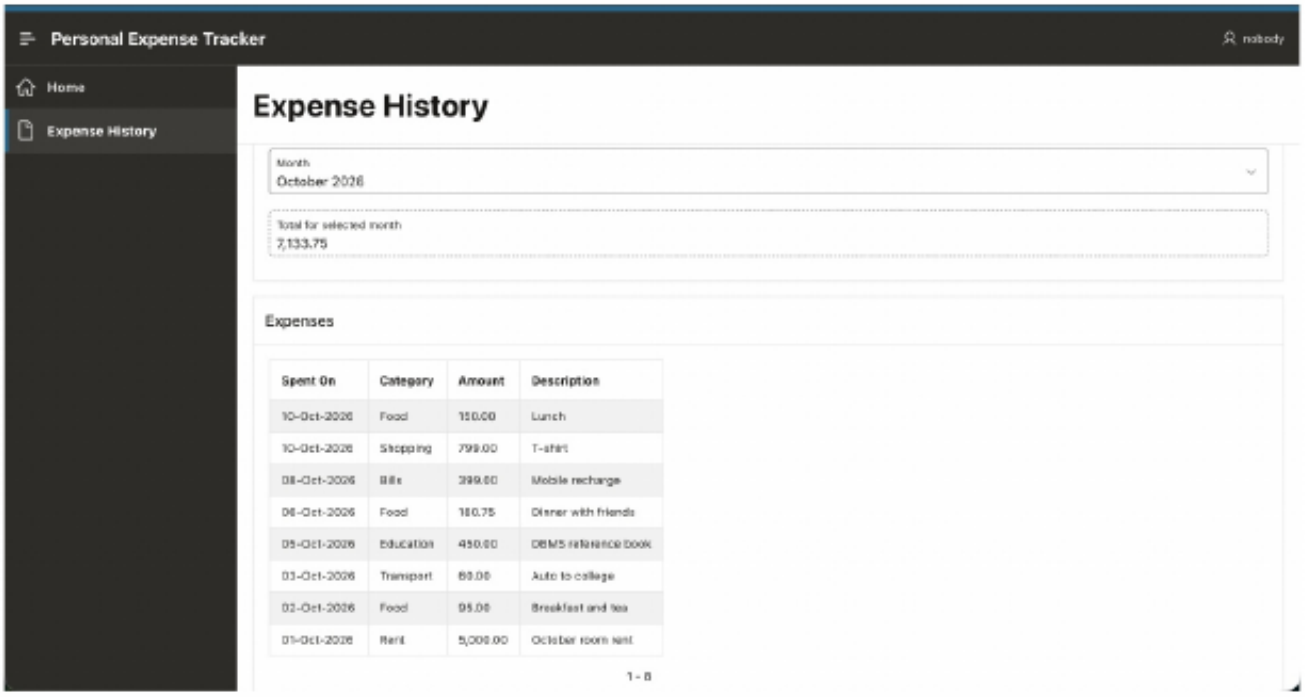


Figure 3. Expense History for October 2026: eight listed expenses and a displayed total of 7,133.75.

Application URL: https://oracleapex.com/ords/r/ujjals_space/personal-expense-tracker/home

Chapter 4: Testing and Results

Testing evidence below separates the supplied live screenshot from the student's reported script run. No additional test execution is claimed in this report.

ID	Test	Observed result	Evidence
T1	Run the database setup script.	Student-reported: 25 SQL/PLSQL statements completed with 0 errors.	Student's supplied test notes.
T2	Add 150.00, Food, 10-Oct-2026, Lunch.	The row is visible in Expense History; October total is 7,133.75.	Supplied history screenshot.
T3	Submit amount -150.	Student-reported message: "Amount must be greater than zero." No row is saved.	Supplied test notes. The form screenshot is pre-submit only.
T4	Select October 2026 in history.	Eight rows are shown; displayed total is 7,133.75.	Supplied history screenshot.

4.1 October total check

The screenshot's visible values sum to 7,133.75: 5,000.00 + 95.00 + 60.00 + 450.00 + 180.75 + 399.00 + 799.00 + 150.00. The first seven values are the October sample records in the supplied SQL script; the 150.00 lunch record is the added entry visible in the screenshot.

4.2 Validation coverage

The SQL procedure also contains checks for a missing date, a future date, an unknown category and a description over 200 characters. These rules are present in the code, but only the negative-amount result is reported as exercised in the supplied test notes.

Chapter 5: User Guide and Limitations

5.1 Open the application

Open the application URL in a browser. The Home page is the Add Expense form. A visitor may open Expense History from the left navigation. The submitted link currently opens without requiring a sign-in, so the application data is shared.

5.2 Add an expense

- Enter a positive amount.
- Choose one category from the list.
- Select today's date or an earlier date.
- Enter a short description if useful.
- Select Add Expense. If a rule fails, read the displayed error and correct the entry.

5.3 Review monthly history

Open Expense History, choose the month, and review the total and table rows. The page shows the date, category, amount and description. The October screenshot demonstrates this path with eight rows.

5.4 Current limitations

- There are no user accounts on the public pages, so records are not private or separated by person.
- The current interface does not provide edit or delete controls.
- Categories are predefined; there is no category management page.
- The application has no budget alerts, graphs, export feature or recurring expenses.
- The APEX learning workspace and application should be treated as a demonstration environment, not a production financial system.

5.5 Possible future improvements

A later version could add authenticated users and per-user data ownership, safe edit and delete flows, category maintenance, budget limits, charts, CSV export and clearer confirmation after a save. These are suggestions only; they are not part of the supplied build.

Chapter 5: Conclusion and References

5.6 Conclusion

The project connects Oracle database programming with a working browser interface. The two-table design records expenses against categories, while a stored procedure validates new entries and a stored function calculates totals by calendar month. Oracle APEX exposes the form and history report as user-facing pages. The supplied screenshots show the public form and a live October history with a total of 7,133.75 across eight visible entries.

The current version meets its classroom purpose and provides a clear base for learning relational design, constraints, stored program units, date filtering, aggregates and APEX page processes. Its public shared access is also its main practical limitation: it should not contain private financial data until access control and data ownership are implemented.

References

1. Oracle. PL/SQL Language Reference, "Overview of PL/SQL." https://docs.oracle.com/en/database/oracle/oracle-database/26/lnpls/overview.html

2. Oracle. PL/SQL Language Reference, "Raising Exceptions Explicitly." https://docs.oracle.com/en/database/oracle/oracle-database/26/lnpls/raising-exceptions-explicitly.html

3. Oracle APEX. Getting Started. https://www.oracle.com/apex/getting-started/

4. Oracle APEX 26.2 Documentation. About the Create Application Wizard. https://docs.oracle.com/en/database/oracle/apex/26.2/htmdb/about-the-full-create-application-wizard.html

5. Student-provided source files: expense_tracker.sql and APEX application export f185095.sql.

6. Student-provided application screenshots and live application URL, accessed 11 October 2026.

Project access

Personal Expense Tracker: https://oracleapex.com/ords/r/ujjals_space/personal-expense-tracker/home